Pages and metadata

Include or exclude pages

Use `[pages]` for Typst pages that should be built but should not appear in navigation, such as blog posts or legal pages:

```toml
[pages]
include = ["blog/*.typ", "legal/privacy.typ"]
exclude = ["drafts/**"]
```

Excluded files are also omitted from copied `.typ` source artifacts in the build output directory.

Put the page title in the document and keep website metadata for fields used by listings, routing, and output options:

```typst
#set document(title: [First post])

#metadata((
  date: "2026-06-10",
  tags: ("release", "website"),
)) <website-metadata>

#title()
```

`index.typ` and `404.typ` are always built when present. If `404.typ` exists, *Calepin* writes `404.html`; if PDF rendering is enabled for that page, it also writes `404.pdf`.

Import shared Typst code

Reusable helper functions can live in an ordinary `.typ` file that pages import. Paths behave exactly as they do in plain Typst: a relative path resolves from the directory of the file that writes it, and a path starting with `/` resolves from the website source directory.

```typst
#import "doc-utils.typ": callout-box     // next to this page
#import "../doc-utils.typ": callout-box  // one directory up
#import "/doc-utils.typ": callout-box    // website source directory

#callout-box[Reusable helper from a shared file.]
```

The same holds for every other Typst path: `#image("diagram.svg")`, `#csv("data.csv")`, `#bibliography("refs.bib")`, and `#include`. Root-relative paths are the more portable choice for shared assets, since they keep working when a page moves to a different directory.

A helper file that sits inside the website source directory is otherwise treated as a page and rendered into the site. Exclude it so it is only ever imported:

```toml
[pages]
exclude = ["doc-utils.typ"]
```

Excluded files stay on disk and remain importable; they are only removed from the built site.

Single documents compiled with `calepin compile paper.typ` follow the same rules, with one difference: their root is the directory that contains the document, so `/` refers to that directory rather than to a website source directory, and a lone document cannot reach files above it with a root-relative path. Relative paths such as `#import "doc-utils.typ"` and `#import "../shared/doc-utils.typ"` work in both cases.

This works because *Calepin* stages the files it hands to Typst next to the document itself: while a page renders, hidden `.calepin-entry.<stem>.*` files appear in its directory, and Typst resolves the

page's relative paths from there. Each build removes the files it staged once its pages have rendered, so a successful build leaves nothing behind.

Two cases leave them in place. A failed render keeps them so that the file paths and line numbers in Typst's error messages still point at readable files, and an interrupted run has no chance to clean up. Both are cleared by the next successful build of the same pages, or by `calepin clean`, which removes `.calepin` directories and stray entry files together. The `.calepin-entry.` prefix is reserved: do not name your own files with it, and add `.calepin-entry.*` to `.gitignore` if you want the occasional leftover kept out of version control.

Page metadata

Add arbitrary page metadata with Typst's `#metadata` function and the `<website-metadata>` label. *Calepin* reads this dictionary while building the site and attaches it to the page entry returned by `calepin.pages()`.

```
#set document(title: [First post])

#metadata((
  date: "2026-06-10",
  tags: ("release", "website"),
  author: "Ada Lovelace",
  summary: "A short release note for the new website.",
  draft: false,
)) <website-metadata>

#title()
```

Use `calepin.pages()` to get structured information about every built page, including its metadata, and process it with normal Typst functions and methods. This is useful for lists, indexes, feeds, publication pages, course pages, and any page that needs to organize other pages in the site.

```
#import "/.calepin/calepin.typ" as calepin
#show: calepin.document

#let posts = calepin.pages()
  .filter(p => p.path.starts-with("blog/"))
  .filter(p => not p.meta.at("draft", default: false))
  .sorted(key: p => p.meta.at("date", default: ""))
  .rev()

#for post in posts [
  - #link(post.href)[#post.title] \
    #post.meta.at("summary", default: [])
]
```

`calepin.pages()` returns one dictionary per built page, excluding `404.typ`. *Calepin* creates these entry fields:

- `path`: source path relative to the website root
- `href`: rendered HTML path relative to the current page
- `title`: resolved page title
- `language`: language code, or `none` when languages are not configured
- `translation_key`: resolved key used to connect translated pages
- `translations`: matching pages in other languages, or an empty dictionary
- `pdf`: PDF path, or `none` when the page has no PDF output
- `excerpt`: short page description, or `none` (see below)
- `meta`: the page's `<website-metadata>` dictionary, or an empty dictionary

`excerpt` saves you from writing a blurb for every page. *Calepin* uses the page's `summary` metadata key if it has one, then `description`, and otherwise derives one from the page itself: the first paragraph of prose, with markup, code chunks, headings, and comments removed, truncated on a word boundary. Web feeds use the same value for their entry summaries.

Only `meta` comes from the page's `#metadata` value. *Calepin* interprets a few optional metadata keys: `title`, `pdf`, `translation_key`, `slug`, `url`, `image`, and `redirect-from`. All other keys are left untouched for your own Typst code.

`image` sets this page's social-card picture and `redirect-from` lists old routes that should redirect here; both are described under website configuration.

There is no required schema for custom metadata. Common keys include `date`, `tags`, `author`, `authors`, `category`, `venue`, `summary`, and `draft`, but you can use any key your page list or template expects. Since `calepin.pages()` returns a Typst array of dictionaries, you can use Typst functions and methods such as `filter`, `map`, `sorted`, `rev`, `at`, and `contains` to select and format the pages you need.

See the tags and taxonomies example for a reusable function that groups this site's documentation pages by metadata value.